

Online Job Portal

Project Overview: Modules & Technology Stack

1. Introduction

An Online Job Portal is a web-based platform that connects job seekers with employers. Employers post vacancies and manage applicants, while job seekers build profiles, upload resumes, search for suitable jobs and track their applications. An admin oversees the whole platform, keeping listings genuine and the system running smoothly.

2. Objectives

- Bring job seekers and employers together on one easy-to-use platform.
- Let candidates search, filter and apply for jobs online and track their status.
- Help employers post jobs, review applicants and shortlist candidates faster.
- Recommend relevant jobs by matching candidate skills with job requirements.
- Give the admin control over users, job listings and platform reports.

3. User Roles

Role	Main Responsibilities
Admin	Approves employers and job posts, manages users and categories, monitors reports.
Employer / Recruiter	Creates company profile, posts jobs, reviews applications, shortlists candidates, schedules interviews.
Job Seeker	Registers, builds profile, uploads resume, searches and applies for jobs, tracks application status.

4. System Modules

The system is divided into independent modules that share one database.

Module	Description	Key Features
1. Login & Authentication	Secure access for all user types.	Separate registration for job seekers and employers, email verification, role-based login, password reset.
2. Admin Dashboard	Central control panel.	User and job statistics, employer approval, manage categories and locations, block or remove spam.
3. Job Seeker Profile	Maintains candidate information.	Personal details, education, skills, experience, resume upload (PDF/DOCX), profile visibility.
4. Employer / Company Management	Maintains recruiter information.	Company profile, logo, description, verification status, recruiter accounts.

Module	Description	Key Features
5. Job Posting Management	Creates and manages vacancies.	Post, edit, close jobs; category, location, salary, skills, experience and deadline.
6. Job Search & Filter	Helps candidates find jobs.	Keyword search, filters by location, category, salary and experience; save jobs for later.
7. Job Application Management	Handles applying for jobs.	One-click apply with resume, application history, status tracking, withdraw application.
8. Applicant Review & Shortlisting	Helps employers select candidates.	View applicants, download resumes, shortlist or reject, schedule interviews, update status.
9. Job Alerts & Notifications	Keeps users informed.	Email alerts for matching jobs, status updates, interview invitations.
10. Job Recommendation (AI)	Suggests jobs to candidates.	Skill-based matching between profile and job requirements, resume keyword parsing.
11. Reports & Analytics	Supports admin and employers.	Jobs posted, applications received, active users, hiring statistics; export to PDF/Excel.

5. Hiring Workflow

1. Employer posts job	2. Admin approves post	3. Job seeker searches & applies	4. Employer reviews & shortlists	5. Interview & selection
------------------------------	-------------------------------	---	---	-------------------------------------

6. System Architecture

The system follows a three-tier architecture:

Frontend React (SPA) web app	Backend FastAPI: auth, business logic and all database access via Prisma	Database MongoDB (backend-managed); resume files on disk/cloud
--	--	--

The React frontend calls the FastAPI backend over REST APIs only. The backend alone owns authentication, authorisation and all database access: it validates the JWT, applies role checks, and reads/writes MongoDB through Prisma Client. Uploaded resumes are stored as files by the backend, with their metadata saved in MongoDB. The frontend holds no direct database or auth logic of its own.

7. Main Database Tables

users, roles, job_seekers, employers, companies, jobs, categories, skills, resumes, applications, saved_jobs, interviews, notifications - all modelled as Prisma schemas and accessed only through the FastAPI backend; the frontend never connects to MongoDB directly.

8. Technology Stack

A commonly used stack for this project is shown below. Alternatives are listed so the stack can be matched to what your course or team prefers.

Layer	Technology (Primary)	Alternatives
Frontend	React (with Vite/CRA), Tailwind CSS or Bootstrap	Next.js, Angular, Vue.js
Backend	Python FastAPI (async REST APIs); owns auth, business logic and all DB access	Django REST Framework, Node.js (Express/NestJS)
Database	MongoDB (document database)	PostgreSQL, MySQL
ORM / Data Access	Prisma Client Python (schema-first ORM for MongoDB)	Beanie, Motor (async MongoDB driver)
API Style	RESTful APIs with JSON, auto-generated docs (OpenAPI/Swagger)	GraphQL
Authentication	JWT issued and verified entirely in FastAPI; password hashing (passlib/bcrypt); email verification	OAuth 2.0, Google / LinkedIn login
File Storage	Resume upload handled by FastAPI (PDF/DOCX), stored on disk or cloud, path saved via Prisma	AWS S3, Cloudinary
Search	MongoDB text indexes with filters	Elasticsearch
AI / Matching	Python (spaCy, scikit-learn) called from the backend for resume parsing and job matching	TF-IDF keyword matching, TensorFlow
Notifications	FastAPI background tasks + SMTP (email alerts)	SMS gateway (Twilio), WhatsApp API
Reporting	Backend-generated exports: openpyxl (Excel), ReportLab (PDF); Chart.js on the frontend	JasperReports
Tools	VS Code, Git & GitHub, Postman, MongoDB Compass, Prisma Studio	Eclipse, GitLab, XAMPP
Deployment	Uvicorn/Gunicorn for FastAPI + Vercel or Docker for React; cloud hosting (AWS/Azure)	Heroku, Render, Railway

9. Benefits

- Saves time for both candidates and recruiters.
- Wider reach: employers find more candidates and candidates see more openings.
- Transparent application tracking from apply to selection.
- Admin control keeps job listings genuine and the platform safe.

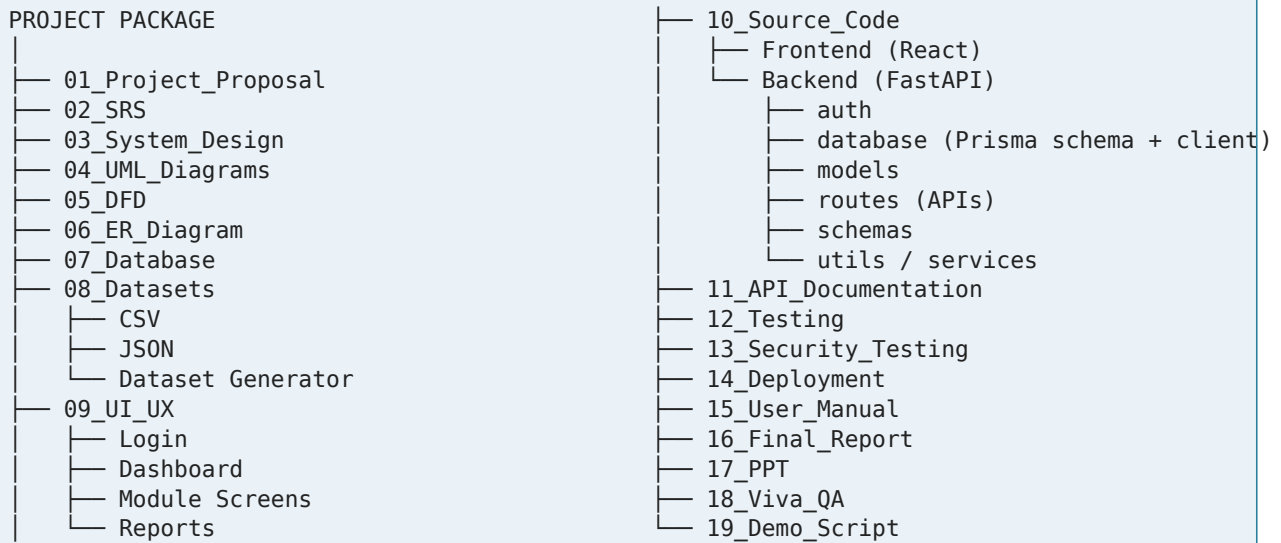
10. Future Enhancements

- Mobile app for job seekers and recruiters.

- AI-based resume screening and ranking of applicants.
- Video interviews and chat between recruiter and candidate.
- Paid featured job listings with a payment gateway.
- Resume builder and skill assessment tests.

11. Project Package Structure

All project deliverables are organised into one package of 19 folders, from planning documents to source code, testing and final presentation.



Folder	Contents
01_Project_Proposal	Problem statement, objectives, scope and feasibility.
02_SRS	Functional and non-functional requirements (SRS).
03_System_Design	Architecture, module design and technology choices.
04_UML_Diagrams	Use case, class, sequence and activity diagrams.
05_DFD	Data flow diagrams (Level 0, 1 and 2).
06_ER_Diagram	Entity-relationship diagram of the database.
07_Database	Schema, SQL scripts and sample data.
08_Datasets	Sample jobs, users and companies in CSV and JSON, plus a dataset generator.
09_UI_UX	Screens for Login, Dashboard, Modules and Reports.
10_Source_Code	Two top-level folders: Frontend (React UI) and Backend (FastAPI). Everything else lives inside Backend: auth (login, JWT, password hashing), database (Prisma schema and client for MongoDB), models, routes (REST API endpoints, including AI-based job matching), schemas (request/response validation) and utils/services (resume parsing, alerts, reports).
11_API_Documentation	REST endpoints, request/response formats, authentication.
12_Testing	Unit, integration and user acceptance test cases and results.
13_Security_Testing	Login, role-access, resume upload validation, SQL injection and XSS tests.
14_Deployment	Server setup, configuration and deployment steps.
15_User_Manual	Step-by-step usage guide for admin, employer and job seeker.
16_Final_Report	Complete project report.
17_PPT	Presentation slides.
18_Viva_QA	Expected viva questions with answers.
19_Demo_Script	Step-by-step flow for the live project demo.